CSE 4095 - Agentic Programming

Hangman REST API Lab | Assignment 13 | October 22, 2026

Exploration observations

My first Easy browser game ended in a loss with the word town. I guessed j, a, e, i, o, m, and l; o was correct, and the six incorrect guesses exhausted the attempts. In the second game, the guessed letters cleared, attempts reset to six, and the word again had four letters. Guessing e revealed the final letter without reducing attempts. Browser evidence was collected in Microsoft Edge Developer Tools; the assignment specifies Chrome.

Q1. Client, server, and game engine

The client is the interface used to play, such as the browser or Python terminal program. It collects guesses, sends requests, and displays the returned game state. The Flask server receives HTTP requests, locates the requested game, and returns JSON responses. The game engine is the server-side logic that manages the hidden word, checks guesses, tracks remaining attempts, and determines whether a game is won or lost.

Q2. Why the hidden word belongs on the server

If the hidden word were sent to the browser at the start, a player could inspect the network response or browser data to read the answer. Keeping it on the server prevents the client from receiving the answer before the game ends. During play, the server returns a masked pattern showing only correctly guessed letters.

Q3. Distinguishing separate games

The server assigns each game a game identifier. Clients include that identifier in URLs when submitting guesses or retrieving state. For example, my browser game used `4faaf996e8c4`, while my curl game used `9c36a30e96ed`. The identifier allows the server to locate the corresponding game state; it identifies a game rather than necessarily identifying a person.

Q4. Game-creation request

- Request URL: `https://hangman-ex89.onrender.com/api/games`
- HTTP method: `POST`
- JSON request body: `{"difficulty":"easy"}`
- Response status: `201 Created`

The status indicates that the request successfully created a new game. The creation screenshots show the request headers, payload, and response.

Q5. Game-creation response fields

My browser returned:

```
{
  "game_id": "4faaf996e8c4",
  "pattern": "_ _ _ _",
  "remaining_guesses": 6,
  "status": "playing"
}
```

- `game_id` identifies the game for later API requests.
- `pattern` represents the word with hidden letters masked. Four blanks indicate four letters.
- `remaining_guesses` is the number of incorrect guesses still allowed.
- `status` indicates whether the game is active or finished. Here, `playing` means it is active.

Q6. Guess request

- Endpoint: `/api/games/4faaf996e8c4/guess`
- Full URL: `https://hangman-ex89.onrender.com/api/games/4faaf996e8c4/guess`
- HTTP method: `POST`
- JSON request body: `{"letter":"e"}`
- Observed response status: `200 OK`

The game identifier in the URL tells the server which game should receive the guess.

Q7. State before and after the browser guess

Before the guess, the pattern was `_ _ _ _`. After guessing e, it became `_ _ _ e`, revealing the last letter. The guess response included `guessed_letters: ["e"]` and `incorrect_letters: []`. These arrays were not included in the creation response; the browser's initial display showed no guessed or incorrect letters.

The remaining guesses stayed at 6 because e was correct. The status stayed `playing` because the word was not fully revealed and the game had not ended.

Q8. GET versus POST

POST requests created a new game and submitted a guess, both of which changed server-side state. GET retrieved an existing game's current state without submitting a guess. In my curl experiment, GET returned the same pattern, guessed letters, and five remaining guesses as the preceding guess response.

Q9. Content-Type header

The `Content-Type: application/json` header tells the server that the request body is JSON. This lets the server interpret fields such as `difficulty` and `letter` as structured request data. The header describes the body being sent; it does not itself create that body or guarantee that its contents are valid.

Q10. Nonexistent game

I requested `/api/games/nonexistent-lab-check` and received:

```
HTTP/1.1 404 Not Found

{"error":"unknown game"}
```

The server received and processed the request, but no game matched the supplied identifier. The 404 response means that the requested game resource could not be found.

Q11. Using a terminal without the web interface

The server exposes an HTTP API that is independent of the browser interface. A terminal program such as curl can send requests to the same endpoints and receive JSON responses. It only needs to follow the API's URL, method, and request-body format. The browser provides a graphical interface, but it is not required for communicating with the game server.

Python Client and Reflection

Q12. Python client implementation

The submitted hangman_client.py creates an Easy game, accepts terminal guesses, displays the server state, and stops on a win or loss. It uses requests with a 30-second timeout and handles connection failures, HTTP errors, and invalid response data. To run it, install requests with `python -m pip install requests`, then run `python hangman_client.py` from the hangman_lab folder. The game engine and hidden word remain on the server.

Q13. Complete-game demonstration

I completed game eb311b9f9325 through the Python client. Correct guesses revealed a, e, and f; incorrect guesses reduced the remaining attempts to zero. A repeated m was rejected locally. The client displayed You lost! and returned to the terminal. The complete transcript appears in the evidence section. The file screenshots/python_client.png is a labeled rendering of that same transcript supplied in chat, rather than a direct terminal screenshot.

Q14. How the client interacts with Flask

My Python client communicates with the Flask server using HTTPS requests. It sends POST requests to create a game and submit guesses. The server returns JSON, which the client parses to display the word pattern, remaining attempts, and game status. The client saves the game identifier returned during creation and includes it in later request URLs so the server updates the correct game. The client also provides a GET function for retrieving state; its main loop uses the updated state returned by each guess.

Q15. AI assistance and prompts

I used AI to organize the assignment into a phased TODO list, implement the Python client, and create test cases. Two prompts I used were "create a todo list with phases that we will complete" and "Try making test cases to test this and I will review at the end". I also used AI to explain the requests and guide me through testing each part. The assistant helped draft the report, and I provided the reflection and reviewed the explanations.

Q16. Development problem and resolution

One problem was a misunderstanding about the development workflow. Because I was concerned about finishing on time, I asked the AI to create tests that I could review afterward. I intended to continue developing in stages, but the assistant interpreted this as permission to complete the implementation and tests together. I noticed that we had moved ahead before I understood each stage, so I asked it to walk me through the completed work. I then tested the browser, API requests, and Python client step by step. In the future, I will specify that the assistant should pause after each phase for my review.

Q17. Replacing human input with a bot

I would replace the terminal input with a function that automatically chooses a letter. The bot could use the returned word pattern, previous guesses, and remaining attempts to choose its next guess. It would submit letters through the same API and continue until the server reported a win or loss. The Flask server could remain unchanged because the bot would follow the existing API and game rules, just as the human-operated client does. Requests would remain sequential and limited to normal gameplay.

Terminal API Evidence

Terminal evidence — User's curl experiment

The following records my successful commands and responses. Commands use PowerShell syntax. An earlier attempt ran curl before assigning BASE_URL and failed with "No host part in the URL"; running the setup commands first resolved it.

```
$BASE_URL = "https://hangman-ex89.onrender.com"
'{"difficulty":"easy"}' | Set-Content -Encoding ascii create_payload.json
curl.exe -sS -X POST "$BASE_URL/api/games" -H "Content-Type: application/json" --data-binary
  "@create_payload.json"

{"game_id":"9c36a30e96ed","pattern":"_ _ _ _ _","remaining_guesses":6,"status":"playing"}

$GAME_ID = "9c36a30e96ed"
'{"letter":"e"}' | Set-Content -Encoding ascii guess_payload.json
curl.exe -sS -X POST "$BASE_URL/api/games/$GAME_ID/guess" -H "Content-Type: application/json"
   --data-binary "@guess_payload.json"

{"guessed_letters":["e"],"incorrect_letters":["e"],"pattern":"_ _ _ _
  _","remaining_guesses":5,"status":"playing"}

curl.exe -sS "$BASE_URL/api/games/$GAME_ID"

{"guessed_letters":["e"],"incorrect_letters":["e"],"pattern":"_ _ _ _
  _","remaining_guesses":5,"status":"playing"}

curl.exe -sS -i "$BASE_URL/api/games/nonexistent-lab-check"

HTTP/1.1 404 Not Found
Content-Type: application/json

{"error":"unknown game"}
```

The last excerpt retains the status, content type, and error body; other response headers are omitted for readability. In this terminal game, e was incorrect, unlike the separate browser game where it was correct.

Complete Python Game

Q13 evidence: actual terminal text supplied in chat.

```
python hangman_lab/hangman_client.py
Game: eb311b9f9325 (easy)

Word: _ _ _ _
Remaining attempts: 6
Enter a letter: a

Word: _ a _ _
Remaining attempts: 6
Enter a letter: e

Word: _ a _ e
Remaining attempts: 6
Enter a letter: m

Word: _ a _ e
Remaining attempts: 5
Enter a letter: d

Word: _ a _ e
Remaining attempts: 4
Enter a letter: f

Word: f a _ e
Remaining attempts: 4
Enter a letter: t

Word: f a _ e
Remaining attempts: 3
Enter a letter: m
You already guessed that letter.
Enter a letter: w

Word: f a _ e
Remaining attempts: 2
Enter a letter: k

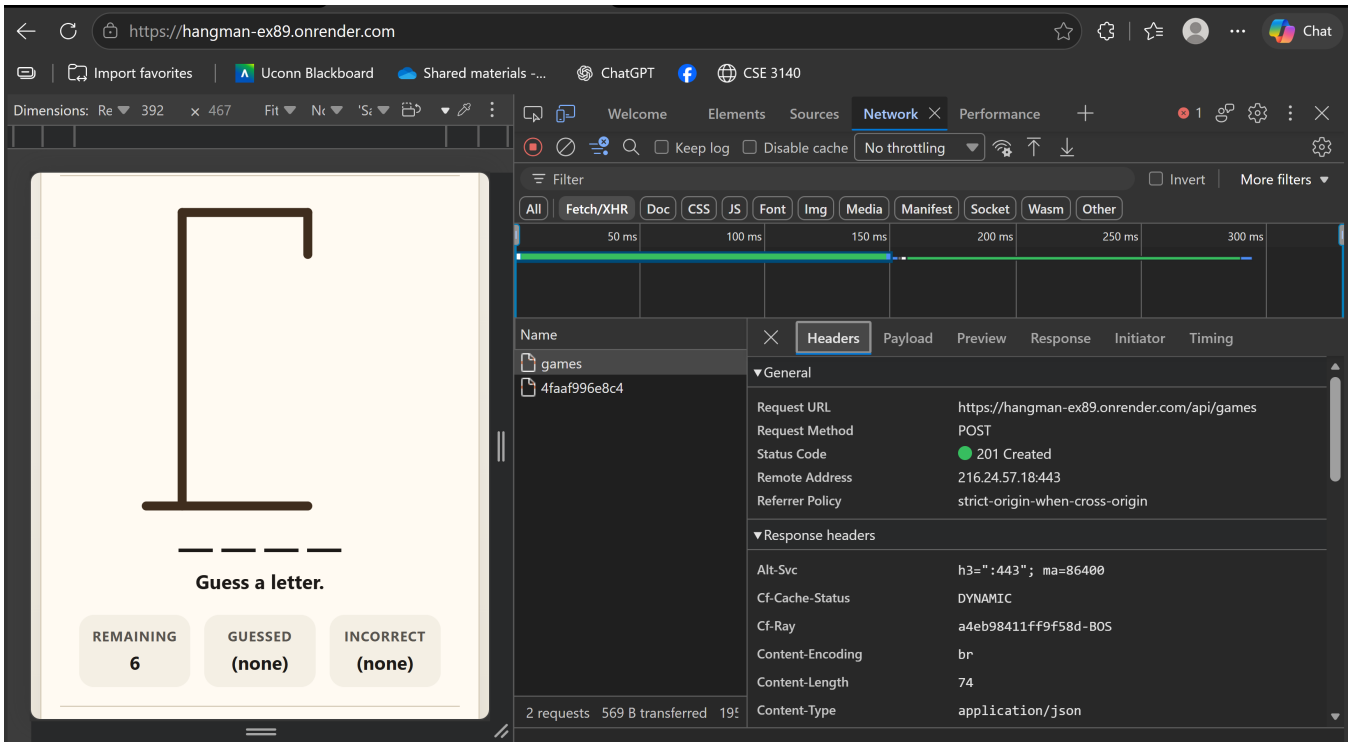
Word: f a _ e
Remaining attempts: 1
Enter a letter: l

Word: f a _ e
Remaining attempts: 0
You lost!
```

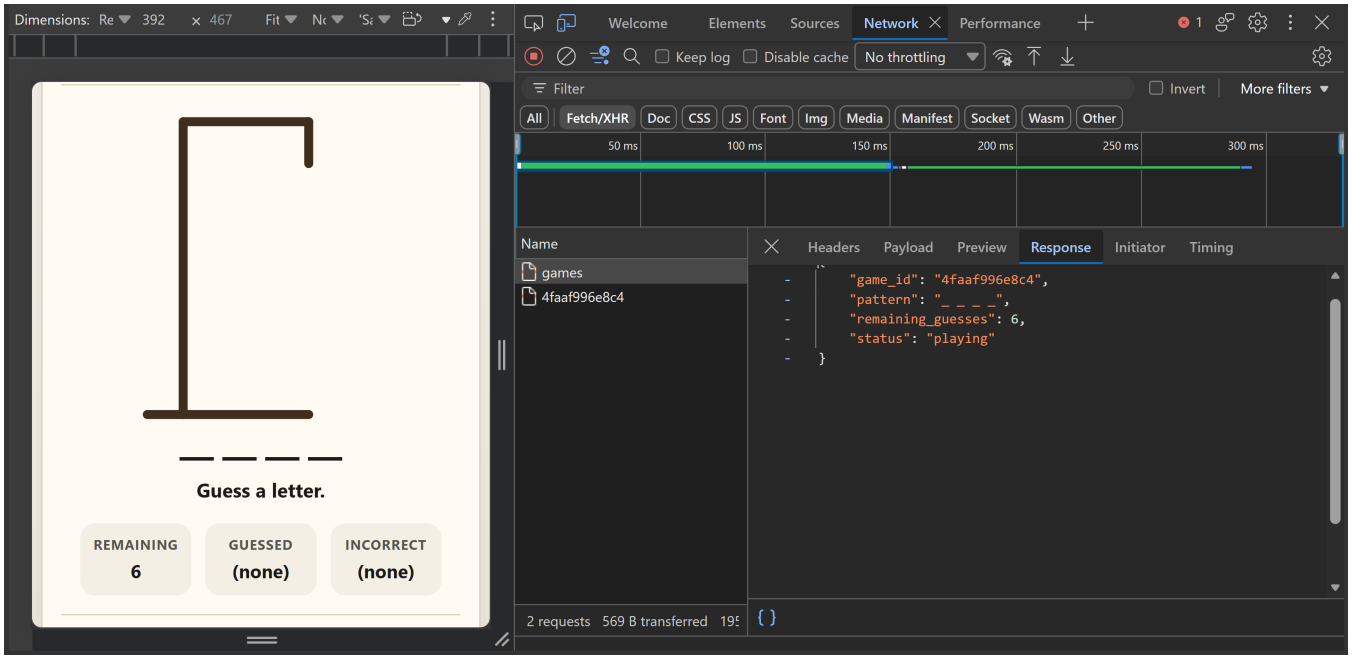
Automated verification

All 16 offline unittest tests passed on my computer (0.239 seconds). The tests exercise request construction, response validation, input handling, simulated wins and losses, and error handling. They mock HTTP traffic and do not replace the live game demonstrated above.

Browser Evidence - Create Game

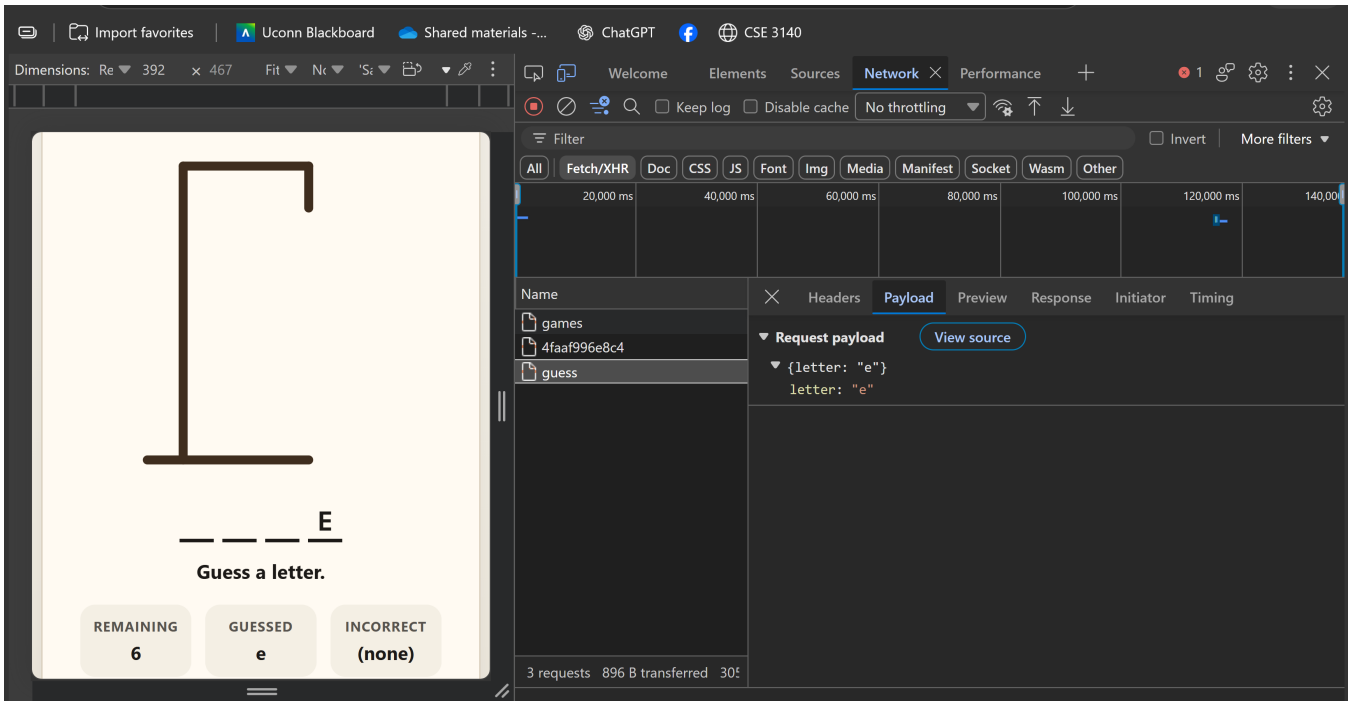


Creation request: POST /api/games returned 201 Created.

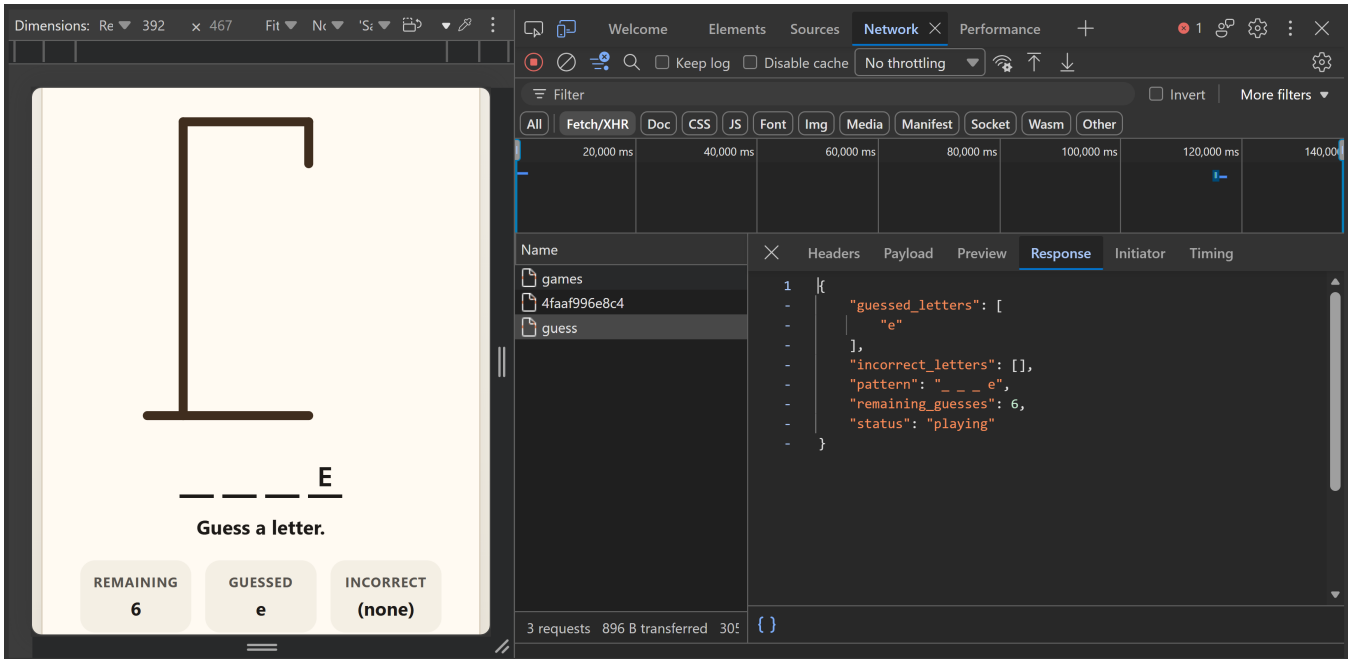


Creation response: identifier, four-letter pattern, six attempts, and playing status.

Browser Evidence - Guess Letter



Guess payload: letter e sent to the server.



Guess response: e revealed, six attempts remain, and game continues.